

wueHans: A Full-Stack Open-Source RISC-V Gaming Console and SoC Architecture

Timo Grundheber, Jonathan Hager, Yannik Stamm

Abstract

To address the lack of hardware sovereignty in proprietary console ecosystems, this paper presents a fully open-source RISC-V gaming platform utilizing a VexRiscv core and Lattice ECP5 FPGA. We implemented a custom SoC featuring dedicated 2D GPU and APU accelerators, supported by a complete LLVM-based toolchain and a high-level Game Development Framework API. Validation through a 48-hour game jam demonstrated the platform's utility, achieving a stable 640×480 at 60 FPS output and high power efficiency for independent development.

Introduction

The global gaming console market is projected to reach approximately 25 billion USD by 2026, with sustained growth expected in the coming years [1]. However, the industry remains an oligopoly dominated by a few major entities, such as Sony, Microsoft and Nintendo [2]. These manufacturers maintain "black box" ecosystems where hardware designs and software stacks are strictly proprietary. This lack of transparency creates high barriers to entry, reducing innovation and limiting choice for both consumers and indie developers who must operate within the restrictive confines of proprietary SDKs and licensing agreements.

While several "open-source" console projects have emerged, most remain reliant on closed-source toolchains or proprietary components, preventing them from being truly transparent [3, 4, 5]. Our project addresses this gap by developing a fully open-source gaming console. Utilizing the ULX3S FPGA board [6] and the RISC-V[7] instruction set architecture (ISA), we have created a platform that prioritizes hardware sovereignty. While the ECP5 FPGA itself and the RAM modules remain commercial components, we utilize a reverse-engineered bitstream tool, Project Trellis [8], to ensure the design remains as open as possible. Furthermore, our architecture includes custom hardware accelerators to optimize gaming performance within an entirely open framework.

System Architecture

The system architecture is built upon the ULX3S FPGA platform, featuring a Lattice ECP5 FPGA with 85K LUTs and 32 MB of SDRAM. At its core, the system utilizes a customizable VexRiscv processor, designed via SpinalHDL, with all internal components integrated through an AXI Lite crossbar interconnect. The global clock speed is set to 50 MHz, with all primary components communicating via a 32-bit AXI

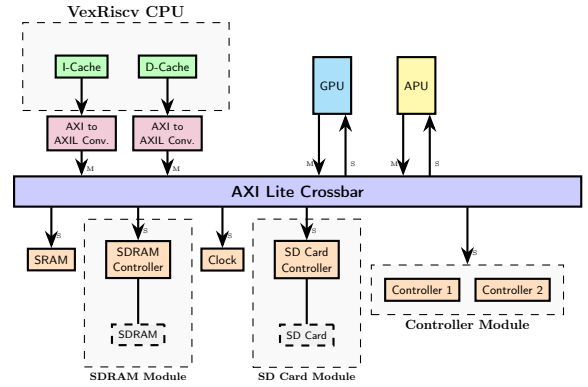


Figure 1: System Architecture

Lite bus. A custom 2D GPU handles rendering via a dual-framebuffer system. It supports colormapping, clearing and swapping the framebuffers and allowing the CPU to direct the drawing of sprites and rectangles. Audio output is managed by a dedicated processing unit capable of mixing eight independent mono channels that can be paired to form stereo channels. Additional hardware peripherals include an SD card for the game and dedicated controller inputs.

On the software side we've developed a complete toolchain based on LLVM and the newlib standard library to provide a modern C++ environment. The deployment process is fully streamlined: the build system prepares the game binary for the SD card, which is then fetched by a custom bootloader at startup. To balance ease of use with performance, we offer a Game Development Framework API. This provides a high level entry point for creators while still allowing direct access for those needing to optimize performance further.

Results

The development process resulted in four fully functional prototype consoles. Each unit integrates the described hardware architecture within a custom de-



Figure 2: *Prototype console with gameplay*

signed, 3D printed enclosure, featuring a dedicated power management system and dual SNES controller for multiplayer functionality.

The system delivers a stable display output of 640×480 at 60 FPS. Hardware utilization on the Lattice ECP5 FPGA is characterized by a moderate LUT footprint (24,206 out of 83,640) and total saturation of available BRAM (208 out of 208 blocks). A key highlight of the hardware is its power efficiency. The console is capable of approximately 10 hours of continuous gameplay on a single charge, significantly outperforming most modern handheld devices and laptops.

To further streamline the development pipeline, a desktop simulator was created, allowing developers to test game logic on a PC before deploying to the physical hardware. To evaluate the software stack and developer experience, we hosted a 48-hour game jam involving three teams of two developers with backgrounds in games engineering and computer engineering. All three teams successfully developed and deployed functional games within the timeframe, validating the ease of use of the Game Development Framework and the stability of the hardware.

Conclusion

This paper presented a fully open-source RISC-V gaming platform that successfully demonstrates hardware sovereignty without sacrificing performance. By integrating custom 2D GPU and APU accelerators with a VexRiscV core, we achieved high-efficiency 640×480

signaling at 60 FPS. The success of the 48-hour game jam confirms that the platform is not only stable but also provides a viable entry point for independent developers.

References

- [1] Ketan Mahajan. “Gaming Console Statistics and Facts (2026)”. In: *Market.us News* (Jan. 2026). URL: <https://www.news.market.us/gaming-console-statistics>.
- [2] Robert A. Lee. “Console Market Share Statistics 2026: Revealed • SQ Magazine”. In: *SQ Magazine* (Aug. 2025). URL: <https://sqmagazine.co.uk/console-market-share-statistics>.
- [3] *Steam Deck™*. [Online; accessed 8. Mar. 2026]. Feb. 2026. URL: <https://store.steampowered.com/steamdeck>.
- [4] *Arduboy*. [Online; accessed 8. Mar. 2026]. Mar. 2026. URL: <https://www.arduboy.com>.
- [5] *Ouya® Console, Game Consol, Game System, Controller Official Website*. [Online; accessed 8. Mar. 2026]. Dec. 2025. URL: <https://theouya.com>.
- [6] *ULX3S*. [Online; accessed 8. Mar. 2026]. Mar. 2026. URL: <https://radiona.org/ulx3s>.
- [7] *RiscV*. [Online; accessed 8. Mar. 2026]. Feb. 2026. URL: <https://riscv.org>.
- [8] *prjtrellis*. [Online; accessed 8. Mar. 2026]. Mar. 2026. URL: <https://github.com/YosysHQ/prjtrellis>.